Predictive Maintenance for Aircraft Engines

Authors: Nishal Karamsetty, Nitin Rayala

Institution: Mahindra University

Date: 8/5/2025

Dataset: NASA CMAPSS (Commercial Modular Aero-Propulsion System Simulation)

Key Methods:
- KMeans Clustering
- Agglomerative Clustering
- Random Forest
- Support Vector Machine
- Ridge Regression
- SMOTE

Abstract:

This project presents a hybrid predictive maintenance approach using the NASA CMAPSS dataset. The primary objective is to classify the degradation stage of jet engines and estimate the time remaining before transitioning to a worse condition. The method begins by clustering time-series sensor data to identify distinct degradation stages. A classifier then predicts the current degradation stage of a unit in operation, and a regression model estimates the time remaining before the unit progresses to the next stage. The risk score combines the predicted probability of failure with the estimated remaining time until failure. It reflects how urgent the need for maintenance is—higher scores indicate both a high probability of failure and a short time remaining before it occurs.

Introduction:

The aviation industry places a high priority on ensuring that aircraft engines are safe and reliable. Engine failures can result in expensive repairs, delays, and even safety risks. To prevent this, we aim to predict engine failure before it occurs. NASA supports this effort through the CMAPSS dataset, which simulates how aircraft engines degrade over time. It includes detailed sensor

data—such as temperatures, pressures, and fan speeds—collected throughout each engine's life, making it ideal for building machine learning models that monitor engine health and predict failures. Our main goal is not just to predict whether an engine will fail, but to identify different stages of failure and create custom labels from the raw data.

Methodology:

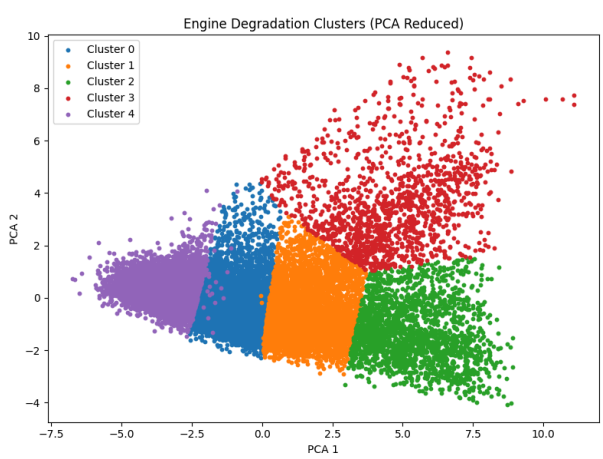
Phase 1:

In Phase 1, we use clustering techniques to analyze sensor data and identify different stages of engine degradation over time. We use two clustering methods: KMeans and Agglomerative Clustering. Our goal is to find five distinct stages of engine degradation.

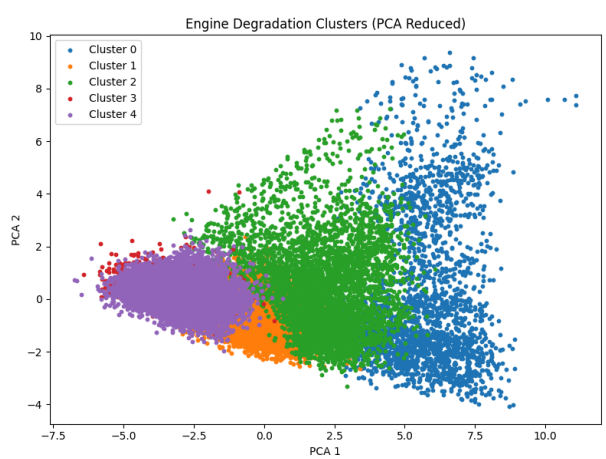
KMeans clustering is an unsupervised machine learning algorithm that groups similar data points together into a predefined number of clusters. It works by initializing several "centroids" randomly, then repeatedly assigning each data point to the nearest centroid using euclidean distance and updating the centroids' positions based on the assigned points. The process continues until the centroids stabilize and the data points are grouped into clusters with similar characteristics.

Agglomerative clustering is a bottom-up hierarchical clustering method. It starts by treating each data point as its own cluster. Then, it repeatedly merges the two closest clusters based on a distance measure (like Euclidean distance) until all points belong to a single cluster or until a specified number of clusters is formed. The merging process is guided by a linkage criterion, such as Ward's method, which minimizes the total variance within clusters. We have added a seed when initializing to select the same centroids/clusters everytime we run the code to avoid results from changing.

KMeans Clustering Graph:



Agglomerative Clustering Graph:



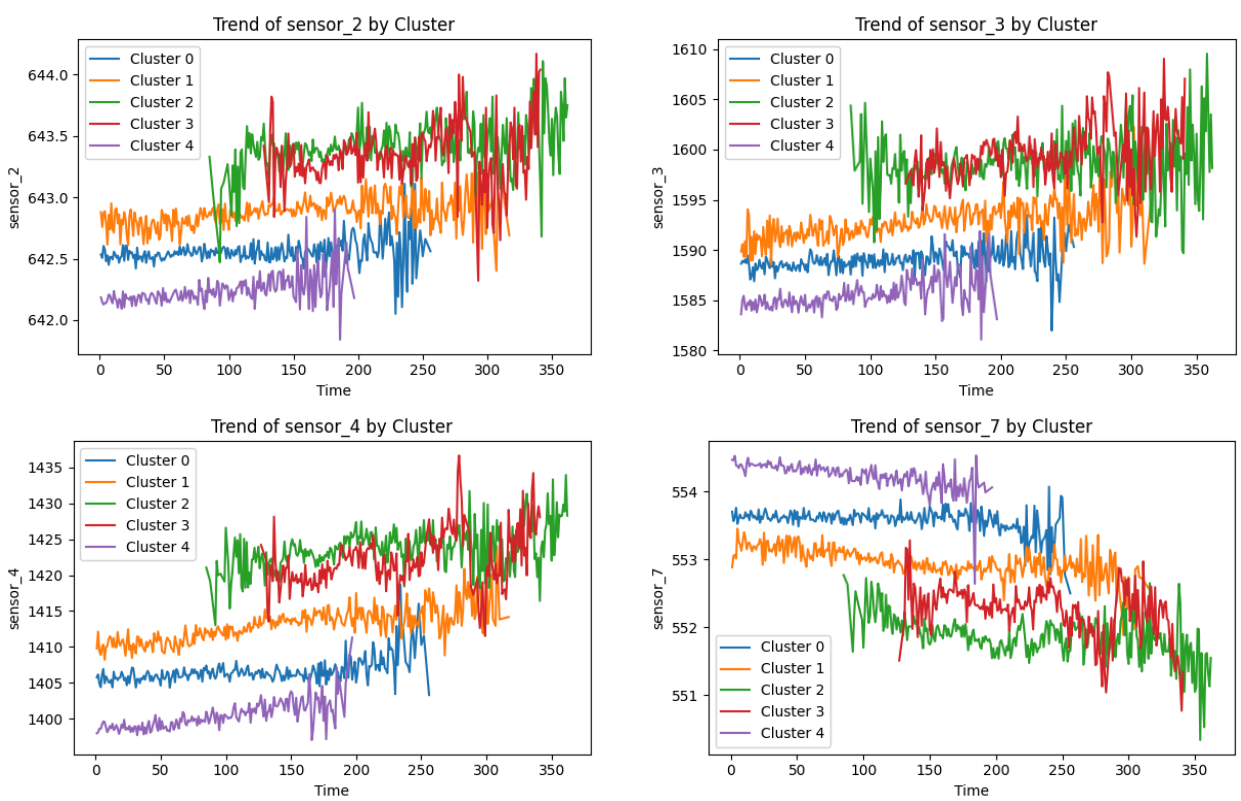
To visualize the clusters, we used Principal Component Analysis (PCA), a dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while retaining most of the important information. The original sensor data had many features, making it difficult to visualize directly. By applying PCA, we reduced the data to two principal components and plotted the clusters in a 2D graph. While the clustering algorithms group the sensor data into five distinct clusters, they do not indicate what each cluster represents in terms of engine health. The clusters are based solely on patterns in the sensor readings, without knowledge of real-world conditions.

Therefore, to manually interpret the meaning of each cluster, we plot graphs showing time vs sensor readings. By visually analyzing trends such as a gradual rise or sudden spikes in sensor values we can estimate which clusters likely correspond to early-stage degradation, moderate wear, or near failure. This manual inspection step is critical because it allows us to link the machine learning output back to meaningful physical stages of engine life.

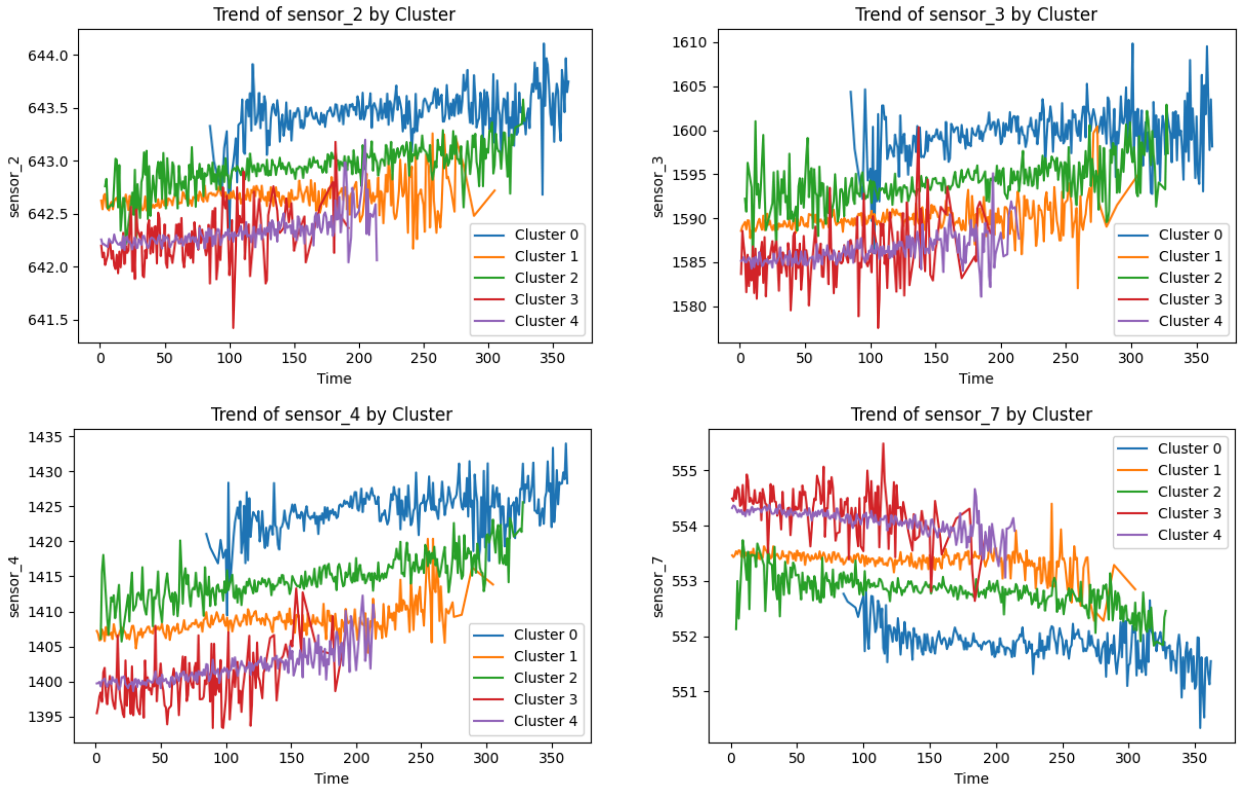
In the CMAPSS dataset, there are 21 different sensor readings available for each engine cycle. However, not all sensors show meaningful changes over time. Many sensors remain relatively constant throughout the engine's life, meaning they do not provide useful information about degradation or failure.

For our analysis, we selectively plotted the trends for Sensor 2, Sensor 3, Sensor 4, and Sensor 7. These sensors were chosen because, based on initial inspection, their readings showed noticeable variation over time, especially as the engine moved closer to failure. By focusing on sensors that show clear trends, we can more accurately map the clusters to physical degradation stages.

KMeans Graphs:



Agglomerative Graphs:



To assign each cluster to a specific degradation stage, we carefully analyzed how sensor readings evolved over time within each cluster. The logic behind the stage definitions is as follows:

- Stage 0 – Normal: Sensor readings stay stable, indicating the engine is healthy.
- Stage 1 – Slightly Degraded: Small, gradual changes suggest early minor wear.
- Stage 2 – Moderately Degraded: Moderate, steady shifts reflect noticeable wear.
- Stage 3 – Critical: Large or rapid changes signal serious issues.
- Stage 4 – Failure: Extremely high or unstable values indicate imminent failure.

Based on this reasoning, we manually assigned clusters to stages for both clustering methods:

- KMeans Clustering:
  Clusters → Stages: {0→1, 1→2, 2→4, 3→3, 4→0}
- Agglomerative Clustering:
  Clusters → Stages: {0→4, 1→2, 2→1, 3→3, 4→0}

Phase 2:

Now that we've matched each cluster to a specific engine degradation stage, we move on to Phase 2: training a model that can predict these stages. We tried three different classifiers — Random Forest, Support Vector Machine (SVM), and Logistic Regression.

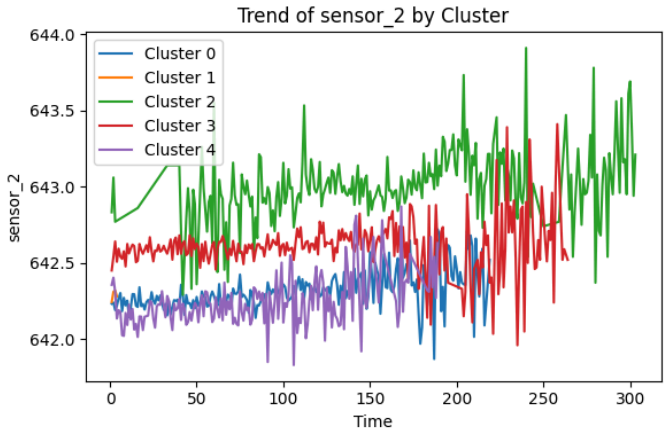
Logistic Regression is a basic method that tries to draw straight lines between classes. It predicts the chance that an engine is in a certain stage based on sensor data. It's fast and easy to use, but it assumes the data follows a simple pattern — which isn't true for our engine sensors. It also doesn't automatically ignore unimportant sensors, so we would need to remove them ourselves.

SVM is more advanced. It can handle more complicated patterns between the data points. But it takes a lot more time to train, especially when there's a lot of data like we have. Also, it needs a lot of fine-tuning — for example, choosing the right type of formula (called a "kernel") to separate the stages correctly. And, unlike Random Forest, it doesn't tell us which sensors are most useful.

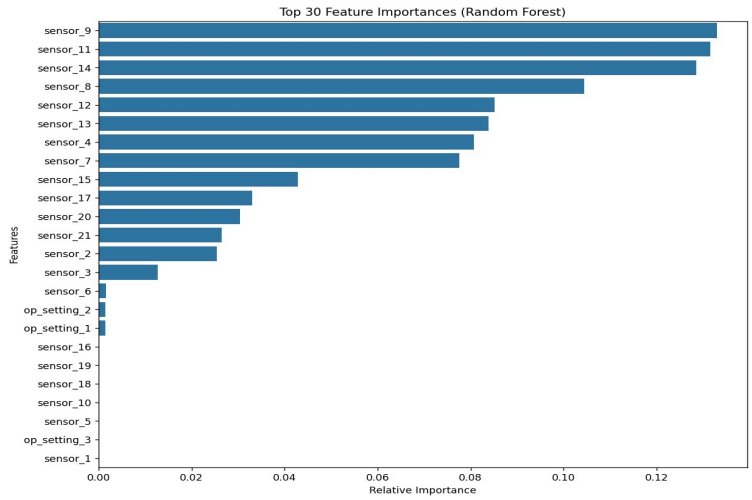
Random Forest works by building many decision trees, each trained on a random part of the data. Each tree makes its own prediction, and the final result is based on what most of them say. This approach helps avoid overfitting and usually gives better accuracy. It also tells us which features are the most important, which helps us understand the model better.

After testing all combinations, we found that using KMeans + Random Forest gave the best results, so we chose this combination. We also used the class_weight='balances' setting while training the model. This makes sure the model doesn't focus too much on the stages that appear more often. It balances things out so that all stages are predicted more fairly.

To help the model understand how sensor values change over time, we added rolling features. These are values like the average or maximum calculated over a sliding window of time. For example, instead of just looking at one reading, we look at the average of the last 5 readings. This helps the model detect trends or sudden changes more clearly. However, this also made cluster 2 very small — only 40 points — so the line for it in the sensor vs. time graph is barely visible.



We also looked at **feature importance** to find out which sensors and rolling features have the biggest impact on the model's predictions. This helped us understand which factors are most useful for detecting engine failure. We showed the results in a bar chart that ranks the top 30 features based on how important they are.



Lastly, we tried another setup: Agglomerative Clustering with SVM. We used Grid Search to find the best settings for the SVM. The accuracy was about the same. But in this setup, cluster 3 had only 243 points, and cluster 2 had over 4,000. To fix this imbalance, we used SMOTE — a method that creates fake data points for the smaller groups by mixing real ones together. This helps the model learn better by giving it more balanced training data.

Phase 3:

In Phase 3 of our engine degradation project, we move from classifying engines into discrete degradation stages to predicting how many time steps (or cycles) remain before each engine reaches its next stage. While Phases 1 and 2 focused on identifying and classifying degradation stages using clustering and a Random Forest classifier, Phase 3 applies supervised regression methods to estimate a continuous value—how much time is left until the next stage. This provides maintenance teams with lead time, enabling them to plan maintenance proactively instead of reacting to breakdowns.

To model this, we can use different regression techniques, including Ridge Regression, Support Vector Regression (SVR), or Random Forest Regressor.

- **Ridge Regression** is a linear model that adds an L^2 regularization term to the usual least squares loss function. This penalty helps reduce overfitting, especially when features are highly correlated or when there are many predictors. The Ridge objective function minimizes:
  $\sum_i (y_i - x_i \cdot \beta)^2 + \alpha \cdot \sum \square \beta \square^2$

Here, $\alpha > 0$ controls how much regularization is applied. Ridge regression is often a good starting point for regression problems due to its balance of simplicity, interpretability, and robustness.

- **Support Vector Regression (SVR)** attempts to fit a function within a margin of error (ε), while minimizing the overall complexity of the model. It can handle nonlinearity by using kernels like RBF (radial basis function) or polynomial. Important parameters include:
    - `C`: penalty for predictions that fall outside the ε-margin,
    - `ε`: the width of the error-tolerance margin,
    - and kernel-specific parameters (like `γ` for the RBF kernel).
- **Random Forest Regressor** is an ensemble method that builds multiple decision trees and averages their predictions. Each tree is trained on a subset of the data, which helps improve accuracy and reduce overfitting. It's well-suited for handling both numerical and categorical features and works well with complex datasets.

To train our regression models, we need target values that indicate how far each data point is from the next stage transition. We compute these by iterating through each engine unit's timeline, grouped and sorted by time. For each record, we check the current degradation stage and scan forward until we find a different stage. We then calculate how many time steps it takes to reach that next stage. If no stage change occurs later, we assign a value of –1, meaning "no upcoming transition." The resulting values are stored in a column called time_to_next_stage.

To keep the model efficient and interpretable, we used only the six most informative sensors as features for training.

We assess the performance of our regression models using three standard metrics:

- **Mean Absolute Error (MAE)**: Measures the average absolute difference between predicted and actual values. It treats all errors equally, making it easy to interpret.
  Formula:
  $MAE = (1/n) \cdot \sum_i |y_i - \hat{y}_i|$
- **Root Mean Squared Error (RMSE)**: Gives more weight to larger errors, highlighting if the model occasionally makes big mistakes.
  $RMSE = \sqrt{[(1/n) \cdot \sum_i (y_i - \hat{y}_i)^2]}$
- **R^2 Score (Coefficient of Determination)**: Indicates how much of the variance in the actual values is explained by the model. An R^2 of 1.0 means a perfect fit, while 0 means the model performs no better than predicting the average.
  $R^2 = 1 - [\sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y})^2]$

Phase 4:

In this phase, we developed a risk assessment system by combining the results from both the classification and regression models. The classification model gave us the probability that an engine is in Stage 4, which is considered the critical failure stage. The regression model estimated how many cycles are left before failure, based on predicted time-to-next-stage values.
To calculate the raw risk score, we multiplied these two outputs:

- High failure probability (from classification)
- Short time to failure (from regression)

This gives us a score that reflects both how likely the failure is and how soon it might happen.
To make the risk scores easier to interpret, we applied min-max normalization, which scales the scores between 0 and 1 based on the lowest and highest values in the dataset. While there are other options (like urgency-based inversion), we used min-max normalization for this project.
Finally, a maintenance alert is triggered if a unit's normalized risk score goes above a threshold (e.g., 0.7). This approach helps prioritize engines that are both at high risk and close to failure, supporting proactive maintenance planning and early warning detection.

Results:

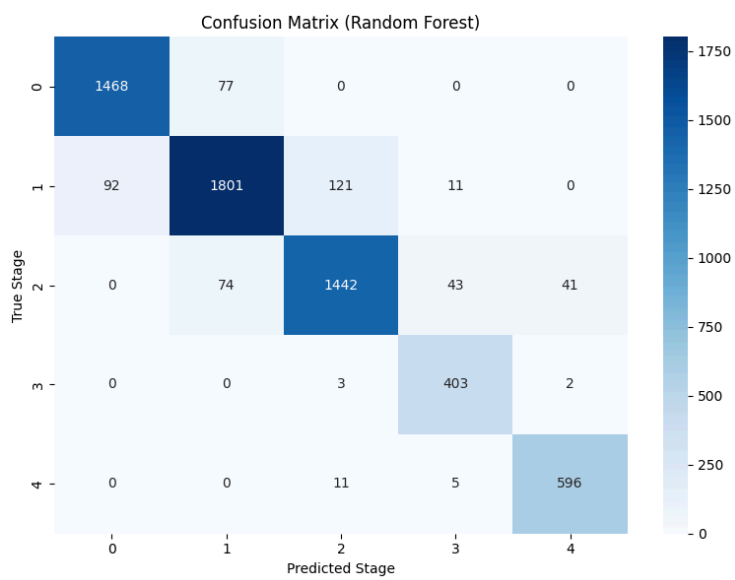
2 models with similar accuracies were created.

Model with KMeans, Random Forest Classification, Ridge Regression:

Training Accuracy: 0.9472
Testing Accuracy: 0.9225

Classification Report (Random Forest):

	precision	recall	f1-score	support
0	0.94	0.95	0.95	1545
1	0.92	0.89	0.91	2025
2	0.91	0.90	0.91	1600
3	0.87	0.99	0.93	408
4	0.93	0.97	0.95	612
accuracy			0.92	6190
macro avg	0.92	0.94	0.93	6190
weighted avg	0.92	0.92	0.92	6190



The classification report provides key metrics to assess our model's performance:

- Support: Indicates the number of samples for each degradation stage in our dataset.
- Precision: Shows what percentage of predictions for each stage were correct (e.g., when the model predicted "Critical," how often it was actually Critical).
- Recall: Measures what percentage of actual cases were correctly identified (e.g., how many truly Critical engines were caught by the model).
- F1-Score: Balances precision and recall, with scores of 93% (Stage 3) and 95% (Stage 4) confirming reliable detection of critical/failure states.

Averages:

- Macro Avg: Treats all stages equally, useful for spotting bias against rare stages.
- Weighted Avg: Accounts for class imbalances by considering sample sizes.

The confusion matrix provides a visual summary of how well the model predicted engine degradation stages by comparing the actual vs. predicted values. Values on the diagonal represent correct predictions (e.g., Stage 0 predicted as Stage 0). Off-diagonal values show misclassifications, such as when the model predicted "Critical" (Stage 3) but the actual stage was "Moderate" (Stage 2).

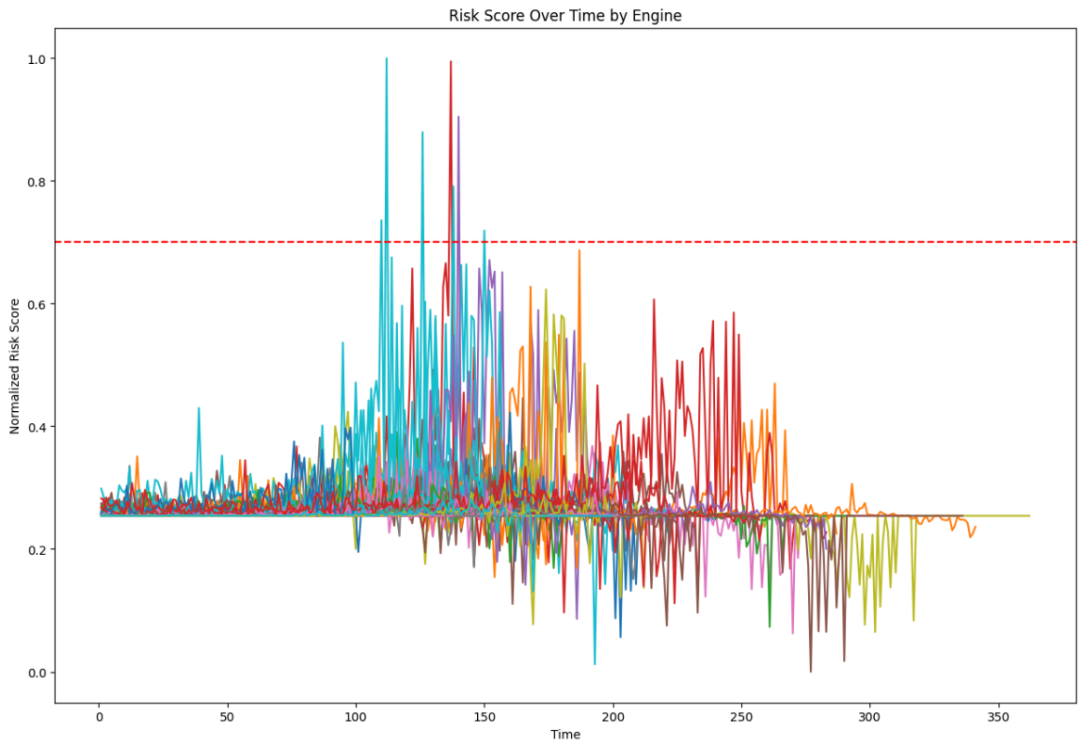
Our classification model achieved an overall accuracy of 92%, with most errors occurring between adjacent stages, like Stage 1 being confused with Stage 2. We also tested SVC and Logistic Regression, but they performed worse than the chosen model.

RUL Regression Results:

- Mean Absolute Error (MAE): 20.63 cycles
- Root Mean Squared Error (RMSE): 27.18 cycles
- R^2 Score: 0.42

Other regression models like Random Forest and SVR were also tested but did not perform as well.

Based on the computed risk scores, the system identified the following engine units as being at high risk: [14, 40, 45, 70]



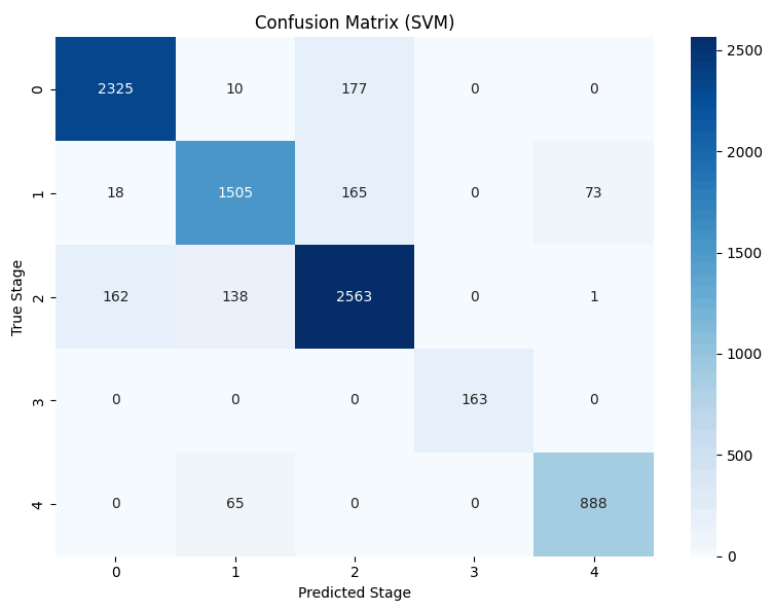
Model with Agglomerative Clustering, SVC, and Random Forest Regression:

Before SMOTE:		After SMOTE:	
2	4296	2	4296
0	3767	0	3767
1	2642	1	2642
4	2000	4	1430
3	1000	3	243

Training Accuracy: 0.9342
Testing Accuracy: 0.9020

Classification Report (SVM):

	precision	recall	f1-score	support
0	0.93	0.93	0.93	2512
1	0.87	0.85	0.86	1761
2	0.88	0.90	0.89	2864
3	1.00	1.00	1.00	163
4	0.92	0.93	0.93	953

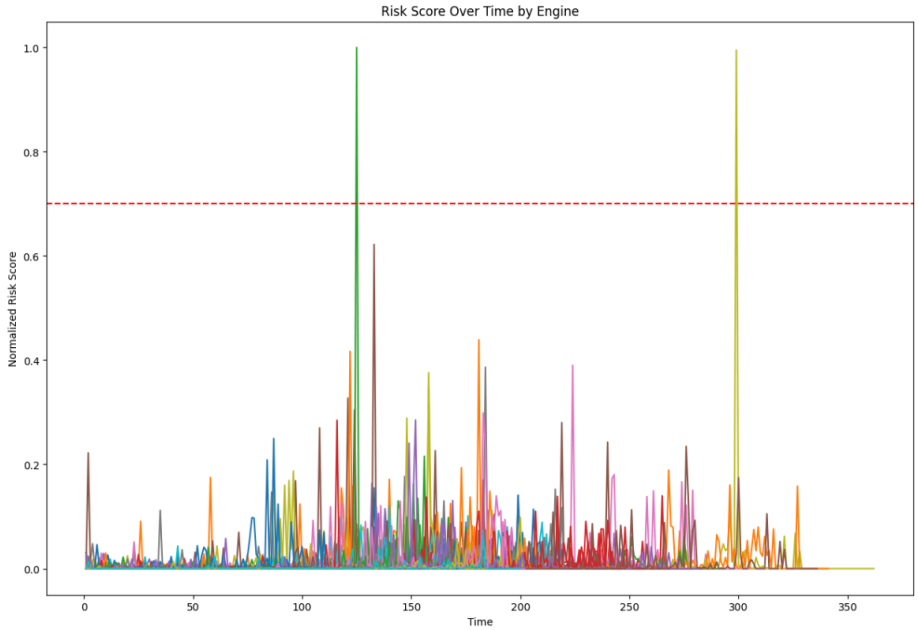


accuracy		0.90	8253	
macro avg	0.92	0.92	0.92	8253
weighted avg	0.90	0.90	0.90	8253

We get an F1 score of 1 as class 3 is very compact and clearly separated from all other clusters. It's positioned tightly in the far left region, which makes it extremely easy for SVM (or any classifier) to distinguish.

RUL Regression Report:
MAE: 14.81
RMSE: 24.45
R2 Score: 0.74
(Other Regression models gave worse accuracies)

Maintenance Alert! Units at risk: [69 93]



We began our experiments using only the FD001 subset of the dataset. However, when we later combined FD001 with FD003, we observed a noticeable drop in model performance—particularly in Phase 4. To maintain higher accuracy and consistency, we chose to focus exclusively on the FD001.

Conclusion:

This project developed a hybrid machine learning pipeline for predictive maintenance by integrating classification and regression models. The classifier estimates the probability of a system entering a failure state (Stage 4), while the regression model predicts time left until Stage 4. These two outputs are then combined in Phase 4 to compute a risk score, which helps prioritize maintenance decisions based on both likelihood and urgency of failure.

Two model variants were evaluated. While Model 1 demonstrated very slightly better classification accuracy, Model 2 significantly outperformed in regression accuracy, meaning it could predict the time to the next degradation stage much more precisely. Since the risk score is a product of failure probability and time to the next stage, accurate regression plays a crucial role in determining when interventions should occur. Therefore, despite the very slight drop in classification precision, Model 2 offers a more reliable and actionable risk assessment due to its superior temporal forecasting capabilities. This makes it better suited for real-world predictive maintenance deployment.